ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ



TRANSVIDCLOUD - NỀN TẢNG XỬ LÝ VÀ PHÂN PHỐI VIDEO TRÊN AWS

BÁO CÁO TIẾN ĐỘ

MÔN HỌC: ĐIỆN TOÁN ĐÁM MÂY

Giảng viên:	TS. Hoàng Xuân Tùng
Mã môn học:	INT3319 1
Thành viên:	Đào Lê Long Vũ - 23021750 Bùi Hải Phương - 23021666 Nguyễn Hữu Lưu - 23021617 Hồ Văn Tiến Nguyên - 23021649

HÀ NỘI - 2026

Mục lục

Mục lục . . . . . **ii**

Danh sách hình ảnh . . . . . **iv**

Danh sách bảng . . . . . **v**

1 Giới thiệu **1**

1.1. Giới thiệu chung về dự án TransVidCloud 1

1.2. Đối tượng người dùng và các bên liên quan 1

1.2.1. Người dùng ứng dụng SaaS trực tiếp 3

1.2.2. Nhà phát triển và đội ngũ kỹ thuật của ứng dụng bên ngoài . . 4

1.2.3. Khách hàng tham chiếu: nền tảng giáo dục trực tuyến 4

2 Giải pháp thực hiện (dự kiến) **6**

2.1. Kiến trúc tính năng của hệ thống 6

2.1.1. Phân hệ quản lý người dùng và xác thực 6

2.1.2. Phân hệ quản lý dự án và tài nguyên video 7

2.1.3. Phân hệ tiếp nhận và lưu trữ video 7

2.1.4. Phân hệ điều phối tác vụ xử lý video 7

2.1.5. Phân hệ xử lý và chuyển mã video 8

2.1.6. Phân hệ quản lý trạng thái và theo dõi tác vụ 8

2.1.7. Phân hệ phân phối và khai thác video đầu ra 8

2.1.8. Luồng xử lý chức năng tổng thể 9

2.2. Kiến trúc triển khai của hệ thống 10

2.2.1. Tổng quan mô hình triển khai 10

2.2.2. Các lớp triển khai 11

2.2.3. Giao diện tương tác giữa các thành phần 13

2.2.4. Chịu lỗi, nhất quán và co giãn 14

2.2.5. Đường triển khai và vận hành 15

2.2.6. Đánh đổi và phương án dự phòng 16

3 Kế hoạch triển khai và ước tính nỗ lực 17

3.1. Cơ sở lập kế hoạch và phân công nguồn lực 17

3.2. Kế hoạch hành động và ước tính nỗ lực 19

3.3. Phụ thuộc, ưu tiên và kết luận kế hoạch 22

3.3.1. Phụ thuộc và các mốc kiểm tra 22

3.3.2. Kết luận kế hoạch 23

Danh sách hình ảnh

1.1 Đối tượng người dùng và các bên liên quan của TransVidCloud 2
1.2 Biểu đồ ca sử dụng của người dùng SaaS trực tiếp 3
1.3 Biểu đồ ca sử dụng của nhà phát triển và ứng dụng bên ngoài 5

2.1 Kiến trúc triển khai TransVidCloud trên AWS. Nét liền: giao tiếp đồng bộ; nét đứt: bất đồng bộ hoặc điều khiển; `[n]`: số giao diện trong Bảng 2.1. 11
2.2 Co giãn Worker theo lượng công việc tồn đọng. Chỉ số `BacklogPerInstance` được tính theo công thức (2.1); scale-in protection được bật khi Worker đang transcode. Worker xử lý job từ SQS (Hình 2.1, giao diện [7]) làm backlog giảm, khép kín vòng điều khiển. 15

3.1 Master Timeline triển khai dự án TransVidCloud 23

Danh sách bảng

2.1 Các giao diện tương tác chính giữa thành phần TransVidCloud 12

3.1 Ước tính năng lực triển khai của nhóm TransVidCloud 17
3.2 Vai trò và trách nhiệm của các thành viên 18
3.3 Kế hoạch hành động tổng hợp cho dự án TransVidCloud 19
3.4 Kế hoạch triển khai theo sprint 21

Chương 1

Giới thiệu

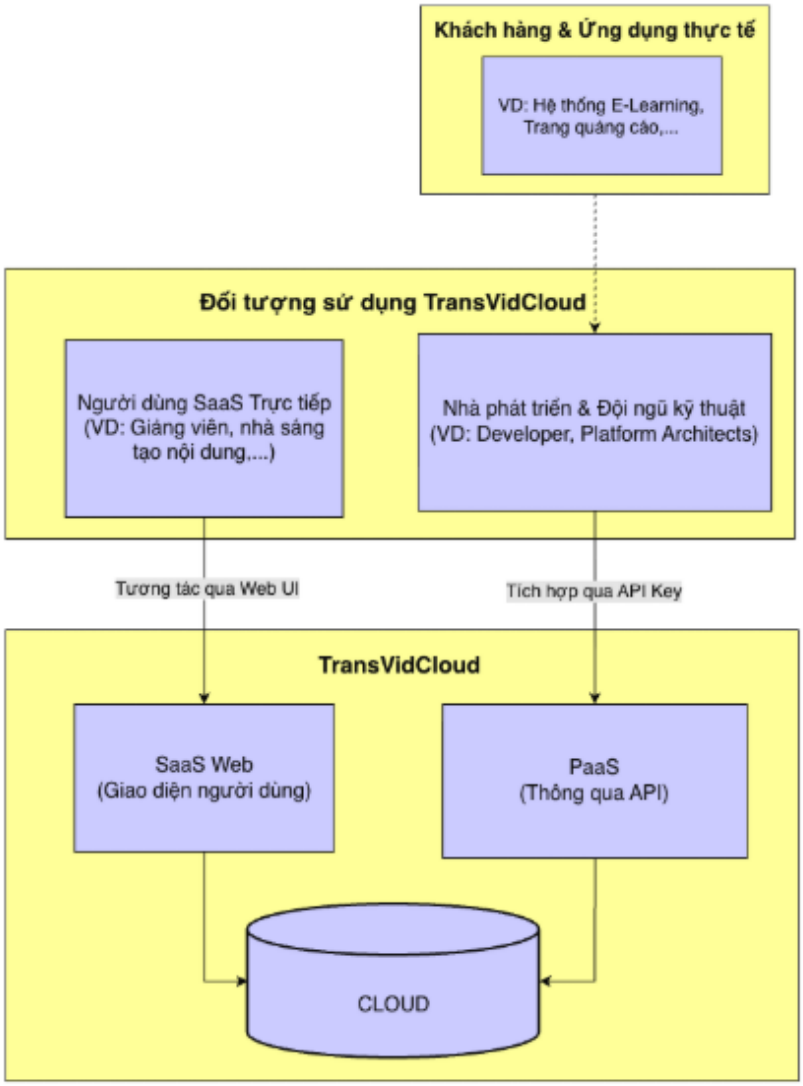
1.1 Giới thiệu chung về dự án TransVidCloud

TransVidCloud là nền tảng xử lý và phân phối video trên nền tảng đám mây, được xây dựng nhằm giải quyết nhu cầu xử lý video có yêu cầu cao về tài nguyên tính toán và lưu trữ. Dự án kết hợp mô hình SaaS dành cho người dùng cuối (các đối tượng cụ thể được trình bày tại Mục 1.2) với Developer API dành cho các hệ thống phần mềm bên thứ ba. Hai phương thức sử dụng cùng vận hành trên một lõi xử lý video phân tán.

TransVidCloud tập trung giải quyết đặc thù của các tác vụ xử lý video, vốn tiêu tốn nhiều CPU và có khối lượng công việc biến động mạnh, có thể tăng đột biến khi nhiều video được tải lên cùng lúc. Hệ thống hỗ trợ tiếp nhận và lưu trữ an toàn video gốc, xử lý bất đồng bộ để tránh gây quá tải cho máy chủ ứng dụng, chuyển đổi video thành nhiều phiên bản có độ phân giải và tốc độ bit khác nhau, đóng gói phục vụ phát trực tuyến như HLS, tạo ảnh thu nhỏ và lưu trữ siêu dữ liệu. Bên cạnh đó, hệ thống cung cấp cơ chế xử lý lỗi và thử lại, kết hợp khả năng tự động co giãn theo khối lượng công việc nhằm tối ưu chi phí tài nguyên, thay vì phải duy trì một cụm máy xử lý video lớn hoạt động liên tục ngay cả khi không có yêu cầu.

1.2 Đối tượng người dùng và các bên liên quan

TransVidCloud hướng tới hai nhóm người dùng chính, với nhu cầu và đặc thù sử dụng khác nhau. Các nhóm đối tượng và mối quan hệ với nền tảng được minh họa trong Hình 1.1.



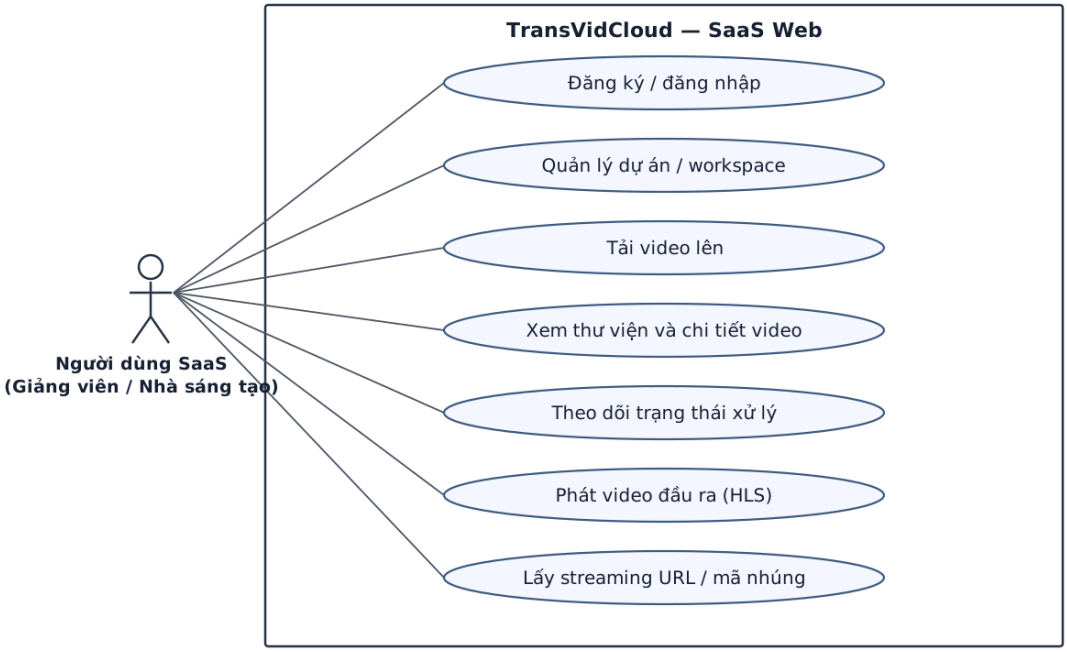
Hình 1.1: Đối tượng người dùng và các bên liên quan của TransVidCloud

1.2.1 Người dùng ứng dụng SaaS trực tiếp

Nhóm người dùng mục tiêu của TransVidCloud gồm các cá nhân và tổ chức có nhu cầu lưu trữ, xử lý và phân phối video trực tuyến, chẳng hạn như nhà sáng tạo nội dung, giảng viên, giáo viên trực tuyến và bộ phận truyền thông số tại doanh nghiệp hoặc tổ chức. Đây là những đối tượng không nhất thiết có kiến thức chuyên sâu về công nghệ đám mây hoặc kỹ thuật xử lý video. Vì vậy, họ cần một nền tảng có giao diện trực quan, thao tác đơn giản và hỗ trợ toàn bộ quy trình từ tải video lên, quản lý nội dung đến theo dõi quá trình xử lý.

TransVidCloud giúp người dùng quản lý và phân phối video trên một nền tảng duy nhất: tổ chức video theo dự án hoặc mục đích sử dụng, tải lên và lưu trữ video dung lượng lớn, theo dõi trạng thái xử lý, xem trước và khai thác video đầu ra. Người dùng có thể sử dụng các phiên bản video với chất lượng khác nhau, lấy đường dẫn phát trực tuyến hoặc mã nhúng để tích hợp vào website, bài viết và các nền tảng khác. Qua đó, nền tảng giảm sự phụ thuộc vào máy tính cá nhân trong quá trình xử lý video, đồng thời đơn giản hóa việc cung cấp nội dung phù hợp với nhiều thiết bị và nhu cầu sử dụng.

Các chức năng chính dành cho nhóm người dùng này được minh họa trong Hình 1.2.



Hình 1.2: Biểu đồ ca sử dụng của người dùng SaaS trực tiếp

1.2.2 Nhà phát triển và đội ngũ kỹ thuật của ứng dụng bên ngoài

Nhóm đối tượng này gồm các kỹ sư phần mềm, kiến trúc sư giải pháp và đội ngũ kỹ thuật vận hành hệ thống web hoặc ứng dụng di động có nhu cầu xử lý, phân phối video. Các hệ thống này có thể thuộc nhiều lĩnh vực, từ thương mại điện tử và mạng xã hội đến cổng thông tin và doanh nghiệp truyền thông. Thay vì tự xây dựng và duy trì hạ tầng xử lý video, họ cần một nền tảng cung cấp API và dịch vụ xử lý video sẵn có, dễ tích hợp vào hệ thống hiện tại và hỗ trợ tự động hóa quy trình từ tải lên, xử lý đến phân phối video.

TransVidCloud cung cấp cơ chế xác thực, tải video trực tiếp lên lưu trữ đám mây, khởi tạo và theo dõi tác vụ xử lý, truy xuất tài sản video đầu ra và nhận thông báo qua Webhook khi quá trình xử lý hoàn tất. Video sau xử lý có thể được tích hợp trực tiếp vào ứng dụng thông qua đường dẫn phát trực tuyến hoặc đường dẫn nhúng. Nhờ đó, nền tảng giúp doanh nghiệp rút ngắn thời gian phát triển, giảm công sức vận hành hạ tầng xử lý video riêng, tối ưu chi phí tài nguyên và hạn chế nguy cơ quá tải hệ thống ứng dụng khi lượng video tải lên tăng đột biến.

Các chức năng tích hợp dành cho nhà phát triển và ứng dụng bên ngoài được minh họa trong Hình 1.3.

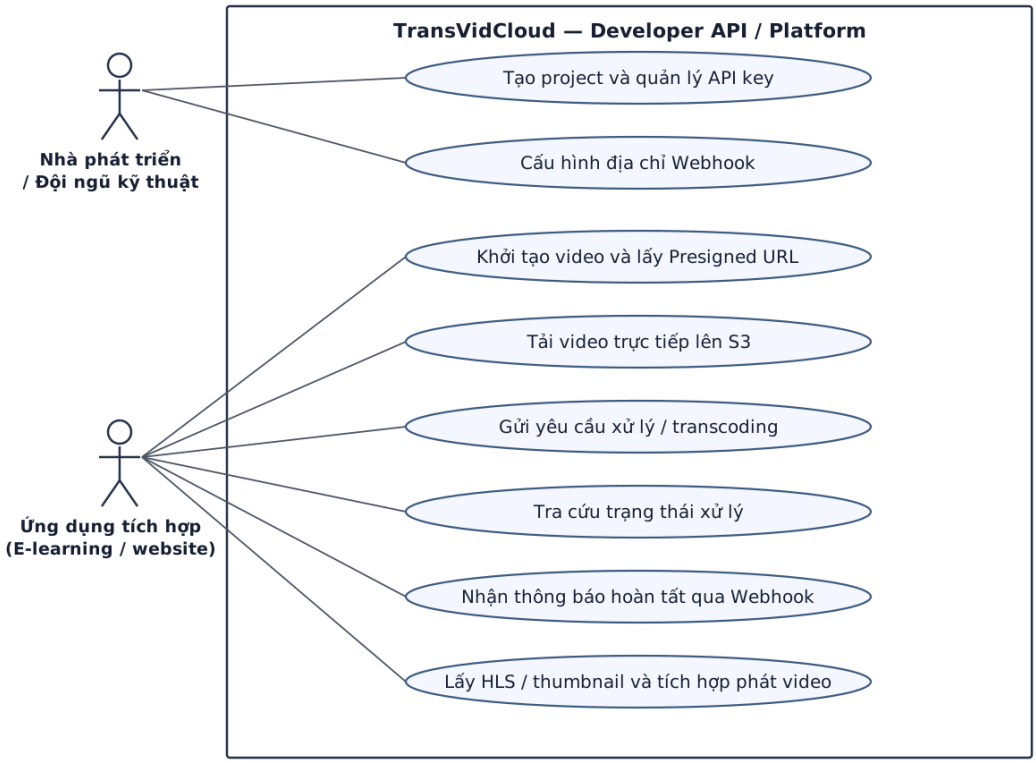
1.2.3 Khách hàng tham chiếu: nền tảng giáo dục trực tuyến

Để minh họa khả năng ứng dụng thực tế của TransVidCloud, một hệ thống đào tạo trực tuyến được sử dụng làm tình huống tham chiếu:

1. Giảng viên sử dụng hệ thống E-learning để đăng tải video bài giảng.

2. Thay vì để máy chủ E-learning tự giải mã và nén video — hoạt động có thể làm nghẽn các dịch vụ khác như làm bài tập và thi cử — hệ thống E-learning gửi yêu cầu xử lý đến TransVidCloud thông qua Developer API.

3. Sau khi được xử lý phân tán trên AWS, video sẵn sàng phát cho nhiều học viên trên trình duyệt và thiết bị di động. Cơ chế phát trực tuyến thích ứng băng thông (*Adaptive Bitrate Streaming*) giúp điều chỉnh chất lượng phát theo điều kiện mạng.



Hình 1.3: Biểu đồ ca sử dụng của nhà phát triển và ứng dụng bên ngoài

Chương 2

Giải pháp thực hiện (dự kiến)

2.1 Kiến trúc tính năng của hệ thống

TransVidCloud được thiết kế theo kiến trúc phân tách các nhóm chức năng. Các nghiệp vụ quản lý người dùng, tiếp nhận yêu cầu và điều phối xử lý được tách biệt với quá trình chuyển mã video tiêu tốn nhiều tài nguyên tính toán. Hệ thống cung cấp hai phương thức tương tác chính: giao diện Web dành cho người dùng SaaS và Developer API dành cho các ứng dụng bên thứ ba. Cả hai phương thức đều sử dụng chung các dịch vụ quản lý và pipeline xử lý video, bảo đảm tính nhất quán về nghiệp vụ và tránh trùng lặp logic xử lý.

Về mặt chức năng, TransVidCloud được tổ chức thành các phân hệ chính như sau:

2.1.1 Phân hệ quản lý người dùng và xác thực

Phân hệ này chịu trách nhiệm quản lý danh tính người dùng và kiểm soát quyền truy cập vào tài nguyên của hệ thống. Đối với người dùng SaaS, hệ thống cung cấp chức năng đăng nhập và xác thực thông qua JWT (JSON Web Token). Đối với các ứng dụng tích hợp bên ngoài, hệ thống hỗ trợ xác thực bằng API Key để cho phép truy cập các chức năng thông qua Developer API.

Các yêu cầu truy cập được xác thực trước khi thực hiện những thao tác liên quan đến dự án, video và tác vụ xử lý. Việc phân biệt hai phương thức xác thực giúp TransVidCloud phục vụ cả người dùng trực tiếp và hệ thống phần mềm bên thứ ba trên cùng một nền tảng.

2.1.2 Phân hệ quản lý dự án và tài nguyên video

Phân hệ quản lý dự án cho phép người dùng tổ chức các video theo từng nhóm nội dung hoặc mục đích sử dụng. Mỗi video được liên kết với một dự án và có bản ghi metadata phục vụ việc quản lý, theo dõi trạng thái và truy xuất kết quả xử lý.

Hệ thống quản lý các thông tin cần thiết như định danh video, dự án sở hữu, vị trí lưu trữ video gốc, trạng thái xử lý và danh sách tài sản đầu ra. Thông qua giao diện Web hoặc API, người dùng có thể truy xuất thông tin video, theo dõi tiến trình xử lý và lấy các tài nguyên được tạo sau khi chuyển mã.

2.1.3 Phân hệ tiếp nhận và lưu trữ video

Phân hệ này chịu trách nhiệm tiếp nhận video đầu vào và quản lý quá trình tải lên. Thay vì truyền toàn bộ dữ liệu video qua máy chủ ứng dụng, hệ thống sử dụng cơ chế Presigned URL để cho phép Client tải video trực tiếp lên dịch vụ lưu trữ đối tượng.

Quy trình bắt đầu khi Client gửi yêu cầu khởi tạo video. Backend tạo bản ghi quản lý và cấp đường dẫn tải lên có thời hạn. Sau khi hoàn tất tải lên, Client gửi yêu cầu xác nhận để hệ thống chuyển video từ trạng thái `UPLOADING` sang `PENDING` và chuẩn bị đưa vào quy trình xử lý.

Cách tiếp cận này giúp giảm tải băng thông và tài nguyên cho máy chủ ứng dụng, đặc biệt khi nhiều người dùng đồng thời tải lên các video có dung lượng lớn.

2.1.4 Phân hệ điều phối tác vụ xử lý video

Phân hệ điều phối chịu trách nhiệm tiếp nhận các video đã tải lên thành công và tạo tác vụ xử lý bất đồng bộ. Mỗi tác vụ được biểu diễn bằng một thông điệp chứa các thông tin định danh cần thiết, bao gồm job, video, project và vị trí lưu trữ video gốc.

Các tác vụ được đưa vào hàng đợi để nhóm Worker tiếp nhận và xử lý độc lập với Backend. Nhờ đó, hệ thống không yêu cầu người dùng duy trì kết nối trong suốt quá trình chuyển mã và không làm thời gian phản hồi của API phụ thuộc vào thời lượng hoặc độ phức tạp của video.

Phân hệ này tách biệt luồng yêu cầu tương tác trực tiếp khỏi luồng công việc xử lý nền, đồng thời tạo điều kiện phân phối tác vụ giữa nhiều Worker khi số lượng video cần xử lý tăng cao.

2.1.5 Phân hệ xử lý và chuyển mã video

Đây là phân hệ cốt lõi của TransVidCloud, đảm nhiệm các tác vụ xử lý video tiêu tốn tài nguyên tính toán. Các Worker hoạt động độc lập, nhận tác vụ từ hàng đợi, truy xuất video gốc và thực hiện xử lý bằng FFmpeg.

Theo phạm vi triển khai dự kiến, pipeline xử lý bao gồm chuyển mã video sang các phiên bản 360p và 720p, đóng gói thành định dạng HLS (HTTP Live Streaming) và tạo thumbnail phục vụ việc hiển thị, xem trước nội dung. Các tài sản đầu ra được lưu vào vùng lưu trữ riêng và liên kết với bản ghi metadata của video tương ứng.

Sau khi xử lý, Worker cập nhật trạng thái và thông tin tài sản đầu ra. Trường hợp tác vụ gặp lỗi, hệ thống hỗ trợ thực hiện lại trong giới hạn cho phép. Các thao tác cập nhật trạng thái được kiểm soát nhằm hạn chế việc xử lý trùng lặp hoặc ghi đè kết quả đã hoàn tất.

2.1.6 Phân hệ quản lý trạng thái và theo dõi tác vụ

Phân hệ này cung cấp khả năng theo dõi vòng đời xử lý của mỗi video, từ khi khởi tạo, tải lên, chờ xử lý, đang xử lý đến khi hoàn tất hoặc gặp lỗi.

Trạng thái của video được lưu trữ tập trung, đóng vai trò là nguồn thông tin thống nhất cho cả giao diện SaaS và Developer API. Backend chịu trách nhiệm cập nhật các trạng thái thuộc giai đoạn tiếp nhận, trong khi Worker đảm nhiệm cập nhật các trạng thái thuộc giai đoạn xử lý.

Người dùng có thể truy vấn trạng thái video thông qua giao diện Web hoặc API để xác định tác vụ đang chờ, đang được xử lý, đã hoàn tất hay gặp lỗi. Cơ chế này giúp tách biệt việc theo dõi kết quả khỏi quá trình thực thi tác vụ, đồng thời hỗ trợ phát hiện và xử lý các trường hợp bất thường.

2.1.7 Phân hệ phân phối và khai thác video đầu ra

Sau khi quá trình xử lý hoàn tất, hệ thống cung cấp các tài sản video đầu ra để người dùng và ứng dụng bên thứ ba khai thác. Các tài nguyên này bao gồm video streaming theo chuẩn HLS, các phiên bản chất lượng khác nhau và thumbnail.

TransVidCloud hỗ trợ truy xuất đường dẫn phát video và tích hợp nội dung vào các website hoặc ứng dụng bên ngoài. Việc phân phối video được tách khỏi máy chủ xử lý nghiệp vụ, giúp hạn chế việc Backend phải trực tiếp truyền tải dữ liệu video đến người xem.

Với định dạng HLS có nhiều mức chất lượng, trình phát tương thích có thể lựa

chọn phiên bản phù hợp với điều kiện mạng và thiết bị của người dùng, hỗ trợ trải nghiệm phát video thích ứng băng thông.

2.1.8 Luồng xử lý chức năng tổng thể

Các phân hệ trên phối hợp với nhau theo một quy trình xử lý thống nhất:

1. **Xác thực và khởi tạo:** Người dùng SaaS hoặc ứng dụng bên thứ ba thực hiện xác thực và gửi yêu cầu tạo video trong một dự án.
2. **Tải video đầu vào:** Backend cấp Presigned URL; Client tải video trực tiếp lên hệ thống lưu trữ và xác nhận hoàn tất.
3. **Khởi tạo tác vụ:** Backend cập nhật trạng thái video thành `PENDING` và gửi thông điệp xử lý vào hàng đợi.
4. **Thực thi xử lý:** Worker nhận tác vụ, chuyển trạng thái sang `PROCESSING`, tải video gốc và thực hiện chuyển mã, tạo HLS cùng thumbnail.
5. **Lưu kết quả:** Worker lưu các tài sản đầu ra, cập nhật metadata và chuyển trạng thái video sang `COMPLETED` khi xử lý thành công.
6. **Truy xuất và phân phối:** Người dùng truy xuất kết quả thông qua Web hoặc API và sử dụng các đường dẫn streaming để phát hoặc tích hợp video.
7. **Xử lý lỗi:** Nếu tác vụ thất bại, hệ thống thực hiện retry theo chính sách đã cấu hình; những tác vụ vượt quá số lần thử được chuyển sang cơ chế xử lý lỗi riêng để theo dõi và cảnh báo.

Kiến trúc chức năng trên bảo đảm cả SaaS Web và Developer API có thể khai thác chung năng lực xử lý video mà không cần xây dựng những pipeline độc lập. Đồng thời, việc phân tách quản lý nghiệp vụ, điều phối và thực thi xử lý giúp TransVidCloud có thể mở rộng năng lực tính toán theo khối lượng công việc, tăng khả năng chịu lỗi và giảm sự phụ thuộc giữa các thành phần.

Các phân hệ chức năng này được ánh xạ sang các thành phần triển khai trên AWS, bao gồm CloudFront, EC2, S3, SQS, DynamoDB và Auto Scaling Group, sẽ được trình bày chi tiết tại mục 2.2.

Kiến trúc chức năng dự kiến của TransVidCloud được minh họa tại Hình 2.1.1. Hệ thống được phân chia thành các nhóm chức năng logic bao gồm lớp tương tác với

người dùng, lớp truy cập dịch vụ, các chức năng nghiệp vụ cốt lõi, lớp xử lý video bất đồng bộ và lớp lưu trữ, phân phối dữ liệu.

Thiết kế hướng tới việc sử dụng chung các chức năng nghiệp vụ và pipeline xử lý video cho cả ứng dụng SaaS và các hệ thống tích hợp thông qua Developer API. Các tác vụ xử lý video được tách khỏi luồng yêu cầu trực tiếp thông qua cơ chế hàng đợi, cho phép các Worker tiếp nhận và thực thi độc lập. Dữ liệu video đầu vào, tài sản sau xử lý và metadata được quản lý tách biệt theo mục đích sử dụng.

Sơ đồ thể hiện phương án tổ chức chức năng ở mức logic, chưa quy định ranh giới triển khai vật lý hoặc công nghệ cụ thể của từng thành phần. Cấu trúc các phân hệ, giao diện tương tác và phạm vi chức năng có thể tiếp tục được điều chỉnh trong quá trình phân tích, thiết kế và triển khai dự án. Phương án ánh xạ các thành phần chức năng sang hạ tầng AWS được trình bày tại mục 2.2.

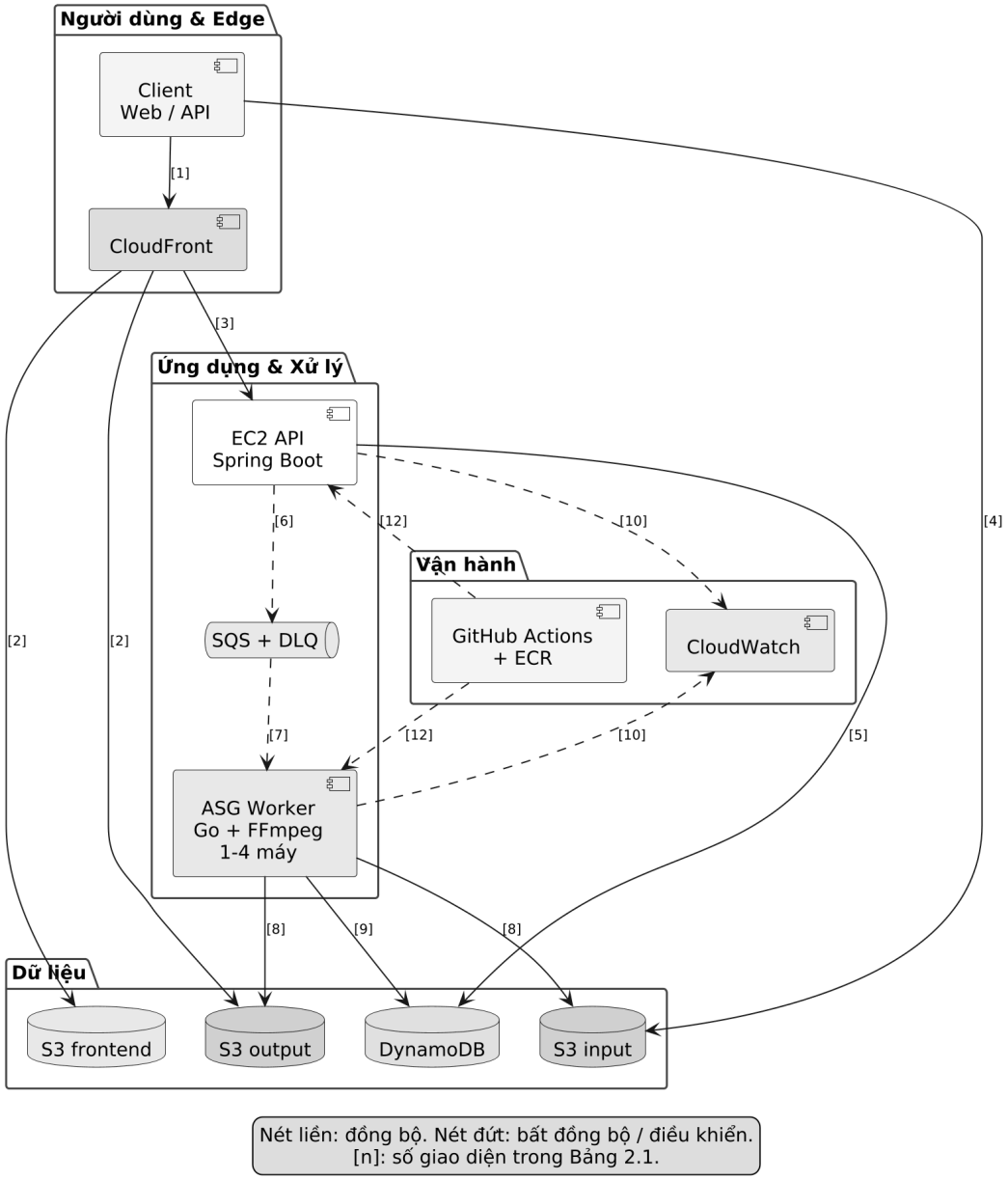
2.2 Kiến trúc triển khai của hệ thống

Kiến trúc triển khai dự kiến ánh xạ các thành phần logic của TransVidCloud xuống hạ tầng AWS và xác định cách các thành phần giao tiếp với nhau. Thiết kế tách các thao tác cần phản hồi ngay (đăng nhập, tải video lên, tra cứu trạng thái) khỏi công việc xử lý nền (chuyển mã video), để người dùng không phải chờ quá trình chuyển mã hoàn tất. Tài nguyên xử lý sẽ co giãn theo khối lượng công việc, quyền truy cập được giới hạn ở mức tối thiểu và toàn bộ hạ tầng có thể tái tạo bằng mã.

2.2.1 Tổng quan mô hình triển khai

Hệ thống theo mô hình *Client – Edge – Application – Processing – Data* (Hình 2.1). TransVidCloud dùng **một backend, hai cách sử dụng**: SaaS Web cho giảng viên và content creator, Developer API cho hệ thống bên ngoài; cả hai dùng chung backend và pipeline xử lý video. Để hình gọn, Hình 2.1 gộp lớp Ứng dụng và lớp Xử lý trong cùng một nhóm; các thành phần như S3 Gateway Endpoint, IAM, SSM, SNS và `metricpub` được mô tả trong Bảng 2.1 và Hình 2.2. Các số `[n]` trong hình tương ứng với cột # của Bảng 2.1.

2.2.2 Các lớp triển khai



Hình 2.1: Kiến trúc triển khai TransVidCloud trên AWS. Nét liền: giao tiếp đồng bộ; nét đứt: bất đồng bộ hoặc điều khiển; `[n]`: số giao diện trong Bảng 2.1.

Bảng 2.1: Các giao diện tương tác chính giữa thành phần TransVidCloud

#	Nguồn → Đích	Loại	Giao thức & mục đích
1	Client → CloudFront	Đồng bộ	HTTPS 443, một domain cho web, `/api/*` và `/media/*`.
2	CloudFront → S3 frontend / S3 output	Đồng bộ	HTTPS GET. Frontend trả React tĩnh; output phát HLS qua OAC, bucket giữ private.
3	CloudFront → EC2 API	Đồng bộ	REST/JSON qua HTTPS, xác thực JWT hoặc API key, theo hợp đồng OpenAPI.
4	Client → S3 input	Đồng bộ	HTTPS PUT trực tiếp bằng presigned URL (SigV4, hạn 15 phút, CORS). API chỉ cấp URL, video không đi qua API.
5	EC2 API → DynamoDB	Đồng bộ	`PutItem`/`UpdateItem`: ghi bản ghi video, chuyển `UPLOADING` → `PENDING`.
6	EC2 API → SQS	Bất đồng bộ	`SendMessage`: message JSON gồm phiên bản schema và định danh job, video, project, khóa file gốc.
7	SQS ↔ Worker (và DLQ)	Bất đồng bộ	`ReceiveMessage` (long polling 20 giây, visibility 300 giây); gia hạn visibility ∼120 giây/lần; `DeleteMessage` = ACK. Quá 3 lần nhận thì chuyển vào DLQ (giữ 14 ngày, có alarm).
8	Worker → S3 input / S3 output	Đồng bộ	AWS SDK qua S3 Gateway Endpoint: `GetObject` file gốc; `PutObject` HLS 360p/720p và thumbnail theo quy ước key, đặt Content-Type chuẩn.
9	Worker → DynamoDB	Đồng bộ	`UpdateItem` có điều kiện: `PROCESSING`, `COMPLETED`, `FAILED_*`, ghi `attemptCount`, `assets`. Video đã `COMPLETED` không bị ghi đè.
10	API, Worker → CloudWatch → SNS	Điều khiển	`PutLogEvents` (log JSON có `jobId`/`videoId`, giữ 7 ngày); alarm DLQ, CPU, độ tuổi message gửi email qua SNS; AWS Budgets 25/50 USD.
11	metricpub → CloudWatch → ASG	Điều khiển	`GetQueueAttributes`, `PutMetricData` (`BacklogPerInstance`); target tracking co giãn 1–4 Worker, có scale-in protection (Hình 2.2).
12	GitHub Actions → ECR → EC2	Điều khiển	OIDC build/test/push image, tag = commit SHA. API deploy qua SSM Run Command; Worker ghi tag vào Parameter Store rồi ASG Instance Refresh.
13	EC2 → SSM Parameter Store	Điều khiển	`GetParameter` đọc cấu hình và secret tại `/tvc/{env}/...` khi khởi động, bằng IAM Role riêng. Chỉ nêu trong bảng, không vẽ ở Hình 2.1.

Lớp người dùng và Edge. CloudFront cung cấp một domain HTTPS với ba behavior: `/` phục vụ giao diện React từ S3 frontend, `/api/*` chuyển tiếp đến EC2 API và `/media/*` phân phối video từ S3 output [1][2][3]. Bucket output giữ private và chỉ đọc được qua CloudFront OAC, nên backend không phải trung chuyển video.

Lớp ứng dụng. Backend Spring Boot chạy trong Docker trên một EC2 `t3.small`

có Elastic IP; Security Group chỉ nhận HTTPS 443 từ CloudFront. API dùng IAM Role riêng theo nguyên tắc *least privilege*. API chỉ tiếp nhận yêu cầu và đưa job vào hàng đợi, không trực tiếp chạy FFmpeg, nên thời gian phản hồi không phụ thuộc độ dài video.

Lớp xử lý. SQS Standard tách API khỏi nhóm Worker (*decoupling*) [6][7]. Worker chạy Go và FFmpeg trong Docker, nhận job bằng long polling, tạo đầu ra HLS 360p, 720p cùng thumbnail rồi lưu vào S3 output [8]. Auto Scaling Group giữ từ 1 đến 4 Worker; hàng đợi hấp thụ lượng job đến dồn dập, còn số máy xử lý thay đổi theo tải (*elasticity*).

Lớp dữ liệu. Ba bucket S3 chia theo mục đích: frontend, input (video gốc) và output (HLS, thumbnail), trong đó input và output đều private. DynamoDB on-demand gồm bốn bảng `users`, `projects`, `apikeys`, `videos`; bản ghi `videos` là nguồn sự thật về trạng thái job. Worker truy cập S3 qua S3 Gateway Endpoint trong VPC nên không cần NAT Gateway.

Lớp vận hành và bảo mật. CloudWatch thu log JSON có `jobId`/`videoId` (giữ 7 ngày) và gửi cảnh báo qua SNS khi DLQ có message, CPU cao hoặc message chờ quá lâu [10]. Cấu hình và secret lưu trong SSM Parameter Store [13]; máy chủ được quản trị bằng SSM Session Manager, không mở SSH. Các IAM Role của API, Worker và CI được tách riêng theo nhiệm vụ.

2.2.3 Giao diện tương tác giữa các thành phần

Bảng 2.1 liệt kê các giao diện chính, là hợp đồng tích hợp giữa nhóm Software và nhóm Infrastructure. Chi tiết endpoint, payload và schema message được tham chiếu tới tài liệu hợp đồng của nhóm Software. Ba giao diện đáng chú ý nhất là:

- **Tải video bằng presigned URL [4].** API cấp một URL có chữ ký, hết hạn sau 15 phút, để Client tải file thẳng lên S3 input. Dữ liệu video không đi qua API, giúp máy chủ ứng dụng không bị quá tải.
- **Gửi job qua SQS [6].** API chỉ gửi một message ngắn chứa định danh job, video, project và vị trí file gốc. Worker chỉ cần hợp đồng message này, không phụ thuộc cấu trúc nội bộ của API.
- **Ghi DynamoDB có điều kiện [5][9].** API phụ trách giai đoạn khởi tạo từ `UPLOADING` sang `PENDING`. Worker chỉ cập nhật có điều kiện các trạng thái xử lý (`PROCESSING`, `COMPLETED`, `FAILED_*`). Ranh giới ghi rõ ràng giúp hai bên không ghi đè trạng thái của nhau.

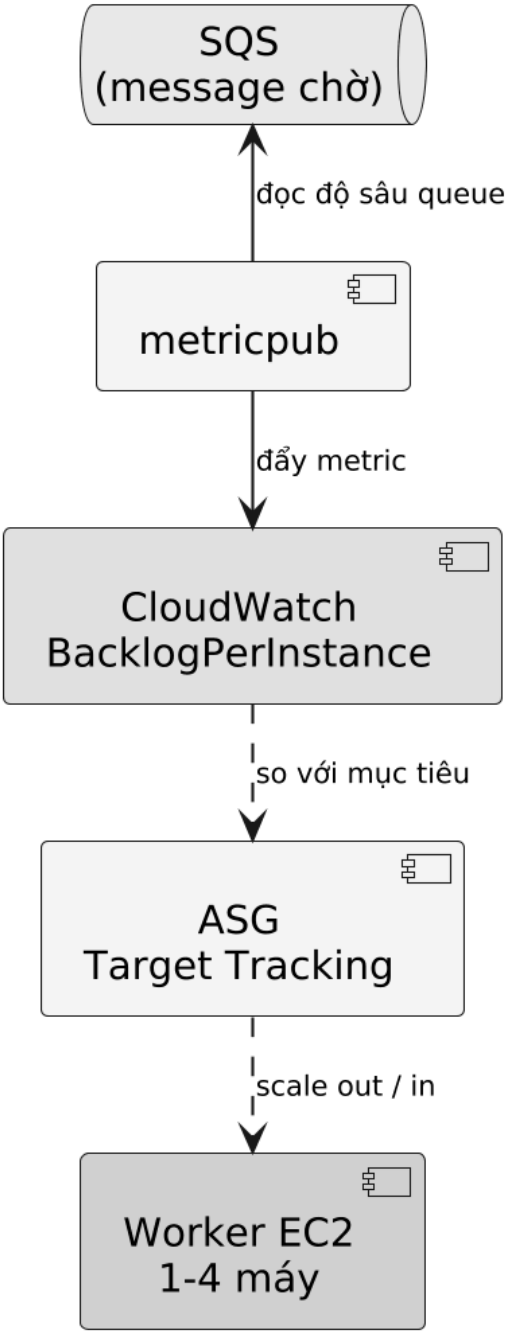
2.2.4 Chịu lỗi, nhất quán và co giãn

Chịu lỗi và nhất quán [7]. SQS Standard giao nhận *at-least-once*. Nếu Worker chết trước khi xác nhận (ACK), message xuất hiện lại sau khi visibility timeout 300 giây hết hạn và Worker khác nhận lại (*fault tolerance*). Với job dài, Worker gia hạn visibility khoảng 120 giây một lần. Message được thử tối đa ba lần trước khi chuyển vào DLQ và phát cảnh báo. Vì job có thể bị giao lại, hệ thống dùng output key ổn định và ghi DynamoDB có điều kiện; video đã `COMPLETED` thì job lặp sẽ bị bỏ qua.

Cơ chế co giãn (Hình 2.2, [11]). Tiện ích `metricpub` đọc số message đang chờ và công bố lên CloudWatch chỉ số:

$$\text{BacklogPerInstance} = \frac{\text{Số message đang chờ}}{\max(1,\ \text{Số Worker})} \tag{2.1}$$

ASG target tracking so chỉ số này với giá trị mục tiêu để tăng hoặc giảm số Worker trong khoảng 1–4. Giá trị mục tiêu sẽ được hiệu chỉnh theo thời gian xử lý thực tế của một job. Scale-in protection được bật khi Worker đang transcode để tránh thu hồi máy giữa chừng.



Hình 2.2: Co giãn Worker theo lượng công việc tồn đọng. Chỉ số `BacklogPerInstance` được tính theo công thức (2.1); scale-in protection được bật khi Worker đang transcode. Worker xử lý job từ SQS (Hình 2.1, giao diện [7]) làm backlog giảm, khép kín vòng điều khiển.

2.2.5 Đường triển khai và vận hành

GitHub Actions lấy quyền AWS tạm thời qua OIDC, build và kiểm thử image, rồi đẩy image API và Worker lên ECR với tag là commit SHA [12]. API được triển khai bằng SSM Run Command; với Worker, pipeline ghi tag image mới vào SSM Parameter Store rồi kích hoạt ASG Instance Refresh. Toàn bộ hạ tầng được quản lý

bằng Terraform (*Infrastructure as Code*) theo quy ước `tvc-{env}-{resource}` tại region `ap-southeast-1`. Hai môi trường *dev* và *demo* dùng chung mã Terraform, chỉ khác biến cấu hình. AWS Budgets theo dõi ngưỡng chi phí 25 USD và 50 USD.

2.2.6 Đánh đổi và phương án dự phòng

- **Mạng:** Worker đặt trong public subnet nhưng không mở inbound, thay cho private subnet kèm NAT Gateway. Cách này tiết kiệm khoảng 32 USD mỗi tháng, đổi lại mức cô lập mạng thấp hơn, nên cần siết quyền IAM và cấu hình định tuyến.
- **CloudFront:** là phương án chính; phương án dự phòng là bỏ CloudFront, cho frontend gọi trực tiếp API và S3 với cấu hình truy cập tương ứng.
- **Số Worker:** duy trì tối thiểu một Worker thay vì scale-to-zero, để luôn có tài nguyên sẵn sàng khi có job mới.

Chương 3

Kế hoạch triển khai và ước tính nỗ lực

3.1 Cơ sở lập kế hoạch và phân công nguồn lực

Kế hoạch triển khai TransVidCloud được xây dựng với bốn thành viên, gồm bốn sprint triển khai, mỗi sprint kéo dài một tuần, và một khoảng thời gian bổ sung để hoàn thiện báo cáo, kiểm thử cuối và chuẩn bị demo. Mỗi thành viên có thể dành khoảng 1–2 ngày làm việc tập trung mỗi tuần cho dự án. Do các thành viên đồng thời phải học tập, làm việc và nghiên cứu công nghệ mới, kế hoạch ưu tiên hoàn thiện luồng xử lý video xuyên suốt trước khi triển khai các chức năng nâng cao.

Nỗ lực được tính theo đơn vị person-day (pd), trong đó 1 pd tương đương một người làm việc tập trung trong một ngày.

Bảng 3.1: Ước tính năng lực triển khai của nhóm TransVidCloud

Thông số	**Giá trị ước tính**	**Diễn giải**
Số thành viên	4	Cùng tham gia phát triển và tích hợp.
Thời gian triển khai	4 tuần	Tương ứng 4 sprint.
Thời gian bổ sung	Khoảng 1 tuần	Hoàn thiện báo cáo, sửa lỗi và chuẩn bị demo.
Capacity thô	20–40 pd	Với mức 1–2 ngày/người/tuần trong 5 tuần.

(Tiếp tục ở trang sau)

Bảng 3.1 (tiếp theo)

Thông số	**Giá trị ước tính**	**Diễn giải**
Capacity danh nghĩa	30 pd	Giả định trung bình 1,5 ngày/người/tuần.
Capacity hữu dụng tham chiếu	Khoảng 24 pd	Sau khi dự phòng 20% cho nghiên cứu, họp, review, tích hợp và sửa lỗi.
Workload baseline đầy đủ	43 pd	Bao gồm 40 pd triển khai và 3 pd báo cáo/demo.

Workload baseline cao hơn capacity dự kiến. Vì vậy, bảng kế hoạch dưới đây thể hiện phương án triển khai đầy đủ theo thiết kế hiện tại, không phải cam kết hoàn thành toàn bộ tính năng trong mọi trường hợp. Nhóm sẽ đánh giá lại sau từng sprint, ưu tiên các hạng mục cốt lõi và hoãn các chức năng nâng cao nếu tiến độ hoặc nguồn lực không đáp ứng.

Bảng 3.2: Vai trò và trách nhiệm của các thành viên

Thành viên	**Vai trò chính**	**Trách nhiệm**
Bùi Hải Phương	Backend Java/Spring Boot	Phát triển Backend và Developer API; xác thực JWT/API key; quản lý project, video và trạng thái job; tích hợp AWS SDK cho Presigned URL, SQS và DynamoDB; phối hợp kiểm thử quyền truy cập tài nguyên.
Đào Lê Long Vũ	Go Worker và xử lý video	Phát triển Worker sử dụng Go và FFmpeg; nhận job từ SQS; chuyển mã video và tạo HLS/thumbnail; triển khai Worker lên EC2; phối hợp kiểm thử scaling và khôi phục khi lỗi.
Nguyễn Hữu Lưu	AWS Foundation và SaaS Web	Thiết lập S3, SQS/DLQ, DynamoDB và cấu hình lưu trữ/mạng nền tảng; phát triển giao diện React tối thiểu cho project, video, trạng thái xử lý, trình phát và mã nhúng.

(Tiếp tục ở trang sau)

Bảng 3.2 (tiếp theo)

Thành viên	Vai trò chính	Trách nhiệm
Hồ Văn Tiến Nguyên	AWS Compute và vận hành	Phụ trách EC2, Auto Scaling Group, CloudWatch, IAM/Security Groups, CloudFront/OAC và quản lý chi phí; triển khai SSM, Terraform và CI/CD nếu còn thời gian.

Việc phân công được tổ chức theo nguyên tắc mỗi module có một người chủ trì, đồng thời các thành viên phối hợp ở những điểm tích hợp. Phương và Vũ vẫn trực tiếp đảm nhiệm một phần công việc Cloud thông qua tích hợp AWS SDK, triển khai Worker và kiểm thử scaling; Lưu và Nguyên phụ trách phần hạ tầng Cloud còn lại.

3.2 Kế hoạch hành động và ước tính nỗ lực

Bảng 3.3 tổng hợp các hạng mục công việc ở cấp module hoặc kết quả bàn giao. Mỗi hạng mục được liên kết với thành phần kiến trúc tương ứng, sprint dự kiến, mức ưu tiên, effort và người chịu trách nhiệm.

P0 là các hạng mục cốt lõi cần ưu tiên hoàn thành. P1 là các chức năng nâng cao có thể hoãn hoặc đơn giản hóa nếu tiến độ không đáp ứng capacity thực tế.

Bảng 3.3: Kế hoạch hành động tổng hợp cho dự án TransVidCloud

ID	Hạng mục công việc	Phần kiến trúc liên quan	Sprint	Ưu tiên	Effort (pd)	Người chịu trách nhiệm
A01	Chốt kiến trúc, hợp đồng tích hợp và khung dự án	SaaS/API, Spring Boot, Worker, SQS, S3, DynamoDB	1	P0	2,5	Phương, Vũ; cả nhóm review
A02	Xây dựng hạ tầng AWS nền tảng và bảo mật mạng	S3, SQS/DLQ, DynamoDB, IAM, VPC, Security Groups, S3 Gateway Endpoint	1, 3	P0	3,0	Lưu, Nguyên
A03	Xây dựng Backend và Developer API	Spring Boot, JWT/API key, Project/Video API, metadata, phân quyền tài nguyên	1–3	P0	5,5	Phương; Lưu hỗ trợ tích hợp

(Tiếp tục ở trang sau)

Bảng 3.3 (tiếp theo)

ID	Hạng mục công việc	Phần kiến trúc liên quan	Sprint	Ưu tiên	Effort (pd)	Người chịu trách nhiệm
A04	Hoàn thiện luồng upload và điều phối job	Presigned URL, S3 Input, DynamoDB, SQS; khởi tạo và xác nhận video	2	P0	2,0	Phương; Lưu và Nguyên hỗ trợ AWS
A05	Xây dựng pipeline Worker và chuyển mã video	Go Worker, EC2, SQS consumer, FFmpeg, HLS 360p/720p, thumbnail, S3 Output	1–2	P0	5,0	Vũ; Phương, Lưu, Nguyên phối hợp
A06	Xử lý lỗi và bảo đảm độ tin cậy của job	SQS/DLQ, retry, visibility timeout, idempotency, cập nhật trạng thái DynamoDB	2	P0	2,0	Vũ, Phương; Lưu hỗ trợ queue
A07	Xây dựng giao diện SaaS tối thiểu	React, Backend API, quản lý project/video, upload và theo dõi trạng thái	1–3	P0	3,0	Lưu; Phương hỗ trợ API
A08	Hoàn thiện phát video và tích hợp nội dung	HLS Player, streaming URL, Embed URL, S3 Output, CloudFront/OAC	3	P0	2,0	Lưu, Nguyên; Vũ kiểm tra đầu ra
A09	Giám sát và tự động co giãn Worker cơ bản	CloudWatch, EC2 Auto Scaling Group, scaling policy, Worker fleet	3	P0	2,5	Nguyên, Vũ
A10	Tích hợp Webhook thông báo trạng thái xử lý	Developer API, Backend và cơ chế callback tới hệ thống ngoài	3	P1	1,5	Phương; Nguyên hỗ trợ cấu hình
A11	Hoàn thiện vận hành và tự động hóa hạ tầng	SSM, alarm/SNS, custom scaling metric, Terraform, GitHub Actions/ECR	3–4	P1	7,0	Nguyên; các thành viên hỗ trợ theo module

(Tiếp tục ở trang sau)

Bảng 3.3 (tiếp theo)

ID	Hạng mục công việc	Phần kiến trúc liên quan	Sprint	Ưu tiên	Effort (pd)	Người chịu trách nhiệm
A12	Kiểm thử tích hợp và thực nghiệm Cloud	Toàn hệ thống; benchmark, queue backlog, throughput, scaling và Worker failure	4	P0	4,0	Vũ chủ trì benchmark; Phương và cả nhóm phối hợp
A13	Hoàn thiện báo cáo và chuẩn bị demo	Kiến trúc thực tế, hướng dẫn triển khai, kết quả thí nghiệm, báo cáo LaTeX và slide	Giai đoạn chốt	P0	3,0	Cả nhóm; mỗi người cập nhật phần mình phụ trách
Tổng effort baseline					**43,0**	

Tổng effort bao gồm 40 pd cho các hạng mục triển khai trong bốn sprint và 3 pd cho giai đoạn báo cáo/demo. Đây là ước tính cho phạm vi đầy đủ, gồm cả các chức năng P1. Các hạng mục được gộp theo module để bảng tập trung vào kết quả công việc chính thay vì từng thao tác cấu hình nhỏ.

Bảng 3.4: Kế hoạch triển khai theo sprint

Sprint	Trọng tâm	Hạng mục triển khai	Đầu ra cần đạt	Effort
Sprint 1	Chốt thiết kế và dựng nền tảng	A01; khởi tạo A02; khung A03; POC A05; khung A07	API contract, message schema và trạng thái job được thống nhất. Các AWS resource cơ bản được tạo; Backend và giao diện có bộ khung chạy được; FFmpeg thử nghiệm thành công với video mẫu.	**8,5 pd**
Sprint 2	Hoàn thiện pipeline xử lý video end-to-end	Hoàn thiện A03, A04, A05, A06 và A07	Video được upload trực tiếp lên S3; job vào SQS; Worker xử lý bằng FFmpeg, tạo HLS 360p/720p và thumbnail; DynamoDB cập nhật trạng thái. Luồng chính hoạt động trên AWS và kết quả truy xuất được qua API.	**11,0 pd**

(Tiếp tục ở trang sau)

Bảng 3.4 (tiếp theo)

Sprint	Trọng tâm	Hạng mục triển khai	Đầu ra cần đạt	Effort
Sprint 3	Hoàn thiện trải nghiệm sản phẩm và khả năng co giãn	Hoàn thiện A02, A03, A07; triển khai A08, A09 và A10 nếu đủ thời gian	SaaS phát được video HLS và lấy mã nhúng; Developer API truy xuất được trạng thái và tài sản đầu ra. Worker có thể scale từ 1 lên 2 máy khi workload tăng và scale-in khi tải giảm. CloudWatch cung cấp metric phục vụ quan sát.	**9,5 pd**
Sprint 4	Hoàn thiện vận hành, kiểm thử và thực nghiệm	A11; A12	Hoàn thành kiểm thử tích hợp, benchmark giữa các cấu hình Worker và một kịch bản Worker gặp lỗi. Thu thập số liệu, biểu đồ. Terraform và CI/CD nâng cao được thực hiện nếu còn thời gian.	**11,0 pd**
Giai đoạn chốt	Báo cáo và chuẩn bị demo	A13	Báo cáo phản ánh đúng kiến trúc triển khai thực tế; có hướng dẫn vận hành, kết quả thực nghiệm và kịch bản demo hoàn chỉnh.	**3,0 pd**
			Tổng effort baseline	**43,0 pd**

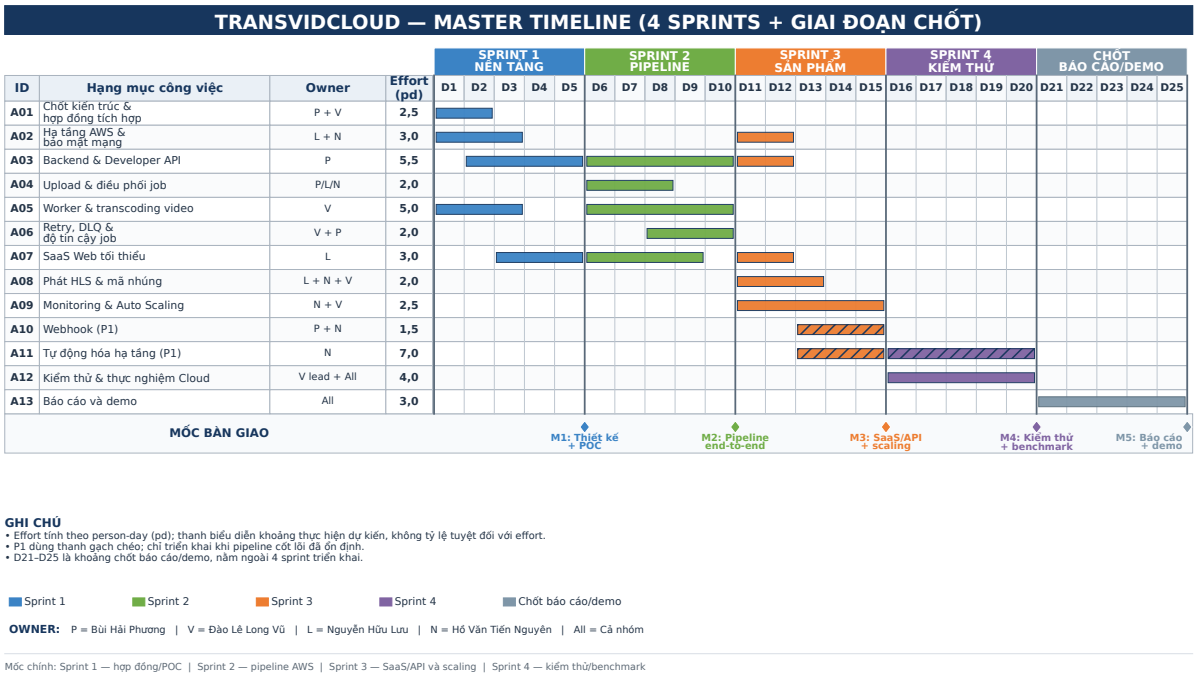
Effort theo sprint được phân bổ từ effort của các hạng mục tại Bảng 3.3; do một số module kéo dài qua nhiều sprint, effort của cùng một hạng mục có thể được chia cho các sprint khác nhau. Các giá trị trên phản ánh khối lượng công việc dự kiến, không phải số ngày lịch.

Để minh họa trực quan trình tự triển khai và sự phân bổ các hạng mục công việc theo thời gian, Hình 3.1 trình bày Master Timeline của dự án TransVidCloud trong bốn sprint. Biểu đồ thể hiện các hạng mục chính, người phụ trách, effort ước tính và các mốc bàn giao dự kiến, qua đó bổ sung góc nhìn tổng quan cho bảng Action Plan và kế hoạch triển khai theo sprint.

3.3 Phụ thuộc, ưu tiên và kết luận kế hoạch

3.3.1 Phụ thuộc và các mốc kiểm tra

Các hạng mục quan trọng của dự án phụ thuộc theo trình tự: thống nhất hợp đồng tích hợp → khởi tạo hạ tầng nền → upload video → gửi job vào SQS → Worker



Hình 3.1: Master Timeline triển khai dự án TransVidCloud

chuyển mã → lưu kết quả → truy xuất và phát video. Auto Scaling và benchmark được thực hiện sau khi pipeline này hoạt động ổn định.

Hai mốc kiểm tra quan trọng được xác định như sau:

- **Cuối Sprint 1:** hợp đồng tích hợp được thống nhất; các POC kỹ thuật quan trọng, đặc biệt là FFmpeg và kết nối với các AWS resource, đã được kiểm chứng.
- **Cuối Sprint 2:** hoàn thành luồng xử lý từ upload video, đưa job vào queue, xử lý bằng Worker, tạo HLS/thumbnail đến cập nhật trạng thái trên AWS. Đây là mốc quan trọng nhất để quyết định có tiếp tục các hạng mục nâng cao hay không.

Nếu chưa đạt mốc cuối Sprint 2, nhóm cần hoãn các hạng mục P1 và tập trung hoàn thiện pipeline chính.

3.3.2 Kết luận kế hoạch

Kế hoạch trên xác định các hạng mục theo module, có liên kết với kiến trúc hệ thống, effort và người chịu trách nhiệm. Trong quá trình triển khai, nhóm sẽ sử dụng các mốc bàn giao cuối mỗi sprint để đánh giá tiến độ và điều chỉnh phạm vi.

Mục tiêu ưu tiên là hoàn thiện một luồng xử lý video end-to-end có thể trình diễn trên AWS, sau đó chứng minh khả năng co giãn tài nguyên và phục hồi khi Worker gặp lỗi bằng các thí nghiệm có số liệu. Các hạng mục nâng cao chỉ được triển khai khi năng lực thực tế cho phép, nhằm bảo đảm sản phẩm cốt lõi, báo cáo và demo được hoàn thành trong thời gian môn học.